

Longest Common Prefix

Problem Statement

Given an array of strings `arr[]`, find the **longest common prefix** shared by all the strings.

If there is no common prefix, return an empty string `""`.

A prefix is a sequence of characters that appears at the beginning of every string.

Example 1

Input

```
arr = ["geeksforgeeks", "geeks", "geek", "geezer"]
```

Output

```
"gee"
```

Explanation

Compare the strings character by character:

```
geeksforgeeks
geeks
geek
geezer
```

The common characters from the beginning are:

```
g e e
```

The next characters are different, so:

```
Longest Common Prefix = "gee"
```

Example 2

Input

```
arr = ["apple", "ape", "april"]
```

Output

```
"ap"
```

Explanation:

```
apple
ape
april
↑↑
ap
```

The first two characters are common to all strings.

Example 3

Input

```
arr = ["dog", "cat", "fish"]
```

Output

```
""
```

There is no character common at the first position.

Approach

The solution uses two main steps:

Step 1: Find the shortest string

The longest common prefix cannot be longer than the shortest string.

For example:

```
["flower", "flow", "flight"]
```

The shortest string is:

```
"flow"
```

Therefore, the common prefix can have a maximum length of **4**.

Step 2: Compare characters vertically

Compare the characters at the same index in every string.

For example:

```
flower  
flow  
flight
```

At index **0**:

```
f  
f  
f
```

All are equal.

At index **1**:

```
l  
l  
l
```

All are equal.

At index 2:

```
o  
o  
i
```

They are different.

Therefore:

```
"f1"
```

is the longest common prefix.

Java Implementation

```
class Solution {  
    public String longestCommonPrefix(String arr[]) {  
  
        int min = Integer.MAX_VALUE;  
  
        // Find the length of the shortest string  
        for (int i = 0; i < arr.length; i++) {  
            if (arr[i].length() < min) {  
                min = arr[i].length();  
            }  
        }  
  
        // Compare characters column by column  
        for (int j = 0; j < min; j++) {  
  
            char ch = arr[0].charAt(j);  
  
            for (int i = 1; i < arr.length; i++) {  
  
                if (arr[i].charAt(j) != ch) {  
                    return arr[0].substring(0, j);  
                }  
            }  
        }  
  
        return arr[0].substring(0, min);  
    }  
}
```

```
}  
}
```

Code Explanation

1. Initialize `min`

```
int min = Integer.MAX_VALUE;
```

`min` stores the length of the shortest string.

Initially, we use the largest possible integer value.

2. Find the shortest string

```
for (int i = 0; i < arr.length; i++) {  
    if (arr[i].length() < min) {  
        min = arr[i].length();  
    }  
}
```

For:

```
arr = ["geeksforgeeks", "geeks", "geek", "geezer"]
```

Lengths are:

```
geeksforgeeks → 12  
geeks         → 5  
geek          → 4  
geezer        → 6
```

Therefore:

```
min = 4
```

We only need to compare the first 4 positions.

3. Outer Loop

```
for (int j = 0; j < min; j++)
```

`j` represents the character position.

For example:

```
j = 0 → first character  
j = 1 → second character  
j = 2 → third character  
j = 3 → fourth character
```

4. Take Character From First String

```
char ch = arr[0].charAt(j);
```

The first string is used as the reference.

For:

```
arr = ["geeksforgeeks", "geeks", "geek", "geezer"]
```

At:

```
j = 0
```

we get:

```
ch = 'g'
```

5. Compare With Remaining Strings

```
for (int i = 1; i < arr.length; i++)
```

We start from `1` because `arr[0]` is already being used as the reference.

Then:

```
if (arr[i].charAt(j) != ch)
```

checks whether the current character is different.

If any string has a different character, the common prefix ends there.

6. Return the Prefix

```
return arr[0].substring(0, j);
```

Suppose:

```
j = 3
```

Then:

```
arr[0].substring(0, 3)
```

returns characters at:

```
0, 1, 2
```

It does **not** include index `3`.

For:

```
"geeksforgeeks"
```

this gives:

```
"gee"
```

7. If All Characters Match

If the loop finishes without finding a mismatch:

```
return arr[0].substring(0, min);
```

This means the entire shortest string is a common prefix.

Dry Run

Consider:

```
arr = ["geeksforgeeks", "geeks", "geek", "geezer"]
```

Step 1: Find minimum length

```
geeksforgeeks → 12  
geeks         → 5  
geek          → 4  
geezer        → 6
```

Therefore:

```
min = 4
```

Iteration 1

```
j = 0
```

Take:


```
arr[0].charAt(0) = 'g'
```

Compare:

```
geeksforgeeks → g  
geeks         → g  
geek          → g  
geezer        → g
```

All match.

Iteration 2

```
j = 1
```

Take:

```
ch = 'e'
```

Compare:

```
geeksforgeeks → e  
geeks         → e  
geek          → e  
geezer        → e
```

All match.

Iteration 3

```
j = 2
```

Take:

```
ch = 'e'
```

Compare:

```
geeksforgeeks → e
geeks         → e
geek          → e
geezer        → e
```

All match.

Iteration 4

```
j = 3
```

Take:

```
ch = 'k'
```

Compare:

```
geeksforgeeks → k
geeks         → k
geek          → k
geezer        → z
```

Mismatch occurs:

```
k != z
```

Therefore:

```
return arr[0].substring(0, 3);
```

Result:

```
"gee"
```

Visual Representation

```
geeksforgeeks
geeks
geek
geezer
^^^
```

The first three characters match:

```
gee
```

At the fourth character:

```
k
k
k
z ← mismatch
```

Therefore:

```
Longest Common Prefix = "gee"
```

Why Find the Shortest String First?

Consider:

```
["hello", "he", "helicopter"]
```

The shortest string is:

```
"he"
```

The common prefix cannot possibly be longer than `2`.

So instead of comparing unnecessary characters, we only check:

```
index 0  
index 1
```

This makes the solution more efficient.

Complexity Analysis

Let:

- n = number of strings
- m = length of the shortest string

Finding shortest string

```
O(n)
```

Comparing characters

In the worst case:

```
O(n × m)
```

Therefore:

```
Time Complexity = O(n × m)
```

where m is the length of the shortest string.

Space Complexity

Only a few variables are used:

```
Space Complexity = O(1)
```

The returned substring itself requires space proportional to the prefix length.

Important Java Methods Used

length()

Returns the length of a string.

```
arr[i].length()
```

Example:

```
"geeks".length() = 5
```

charAt()

Returns the character at a particular index.

```
arr[0].charAt(j)
```

Example:

```
"geeks".charAt(0) = 'g'  
"geeks".charAt(1) = 'e'
```

substring()

Returns part of a string.

```
arr[0].substring(0, j)
```

The starting index is included, while the ending index is excluded.

Example:

```
"geeks".substring(0, 3)
```

returns:

```
"gee"
```

Edge Cases

1. Single String

```
arr = ["hello"]
```

Output:

```
"hello"
```

2. No Common Prefix

```
arr = ["dog", "cat", "fish"]
```

Output:

```
""
```

3. One String Is a Prefix of Another

```
arr = ["flower", "flow", "flight"]
```

Output:

```
"fl"
```

4. Identical Strings

```
arr = ["abc", "abc", "abc"]
```

Output:

```
"abc"
```

Key DSA Pattern

This problem demonstrates **vertical string comparison**.

```
      index
      ↓
geeksforgeeks
geeks
geek
geezer
↑
compare same character position
```

Pattern

1. Find shortest string
↓
2. Start from index 0
↓
3. Take character from first string
↓
4. Compare with every other string
↓
5. If mismatch → return prefix
↓
6. If no mismatch → continue

Final Complexity

Time Complexity : $O(n \times m)$
Space Complexity : $O(1)$

where n is the number of strings and m is the length of the shortest string.

Key takeaway: The common prefix cannot be longer than the shortest string, so first find the shortest length and then compare characters column by column.